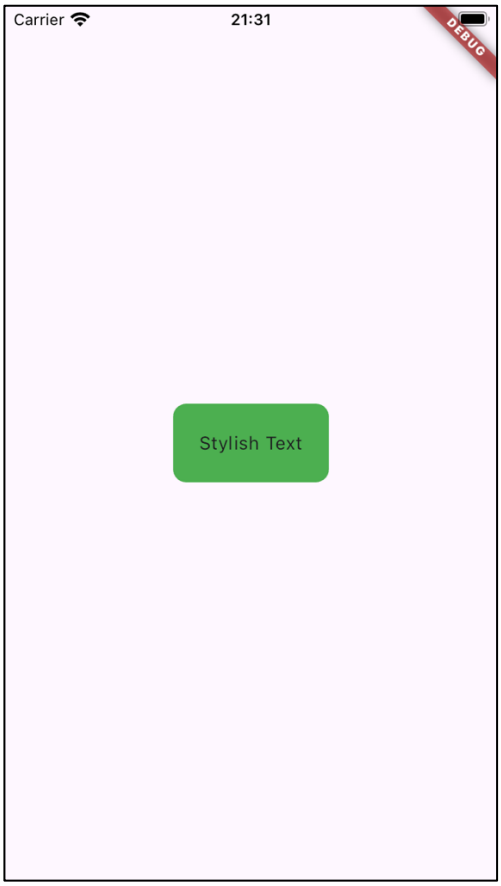

Why Does Flutter Use Functions and Properties Inside Widgets?

a. Why Are Functions and Properties Used in Widgets?
- Flutter allows functions and properties to be passed inside widgets.
- This helps define both appearance (UI) and behavior (logic) in one place.
- Instead of writing UI and logic separately (like older UI frameworks), Flutter keeps them together.

b. Example: Adding Actions with onPressed (Button Example)
- You can pass a function inside the onPressed property of a button widget.
- This function tells the button what to do when clicked.
- Both appearance (UI) and behavior (logic) stay together in one place.
- This is example code :

```
ElevatedButton(
  onPressed: () {
    print('Button clicked!');  // This function runs when
the button is pressed
  },
  child: Text('Click Me'),  // The text shown inside the
button
);
```

- Explanation:
  - onPressed is a property where a function is passed.
  - The function executes when the button is clicked.

- The UI (Text('Click Me')) and logic (onPressed) are together.

◊ Separating Functions for Better Code Organization

- Instead of defining functions inside widgets, you can define them separately and reference them.
- This makes the code cleaner and easier to maintain.
- Useful when the function is complex or used multiple times.
- For example, this is code using separate function :

```
void buttonClicked() {
  print('Button clicked!');  // This function runs when the button is pressed
}

ElevatedButton(
  onPressed: buttonClicked,  // Just reference the function by its name
  child: Text('Click Me'),  // The text shown inside the button
);
```

- Benefits of Separating Functions:
  - ✓ *Reusability* → The same function can be used in multiple places.
  - ✓ *Clarity* → UI code remains clean and focused on design, while logic is handled separately.
  - ✓ *Maintainability* → If the button behavior changes, update only the function, not every button.

c. Example: Using Properties for Styling (TextStyle in Text Widget)

- Properties allow you to customize a widget's appearance.z
- Instead of defining styles separately, you can pass a TextStyle object inside the style property.
- This makes customization more flexible and readable.
- This is example code :

```
Text(
  'Hello, Flutter!',
  style: TextStyle(  // TextStyle customizes the appearance of the text
    fontSize: 24,    // Makes the text larger
    fontWeight: FontWeight.bold,  // Makes the text bold
    color: Colors.blue,  // Changes the text color to blue
  ),
);
```

- Explanation:
  - style is a property that takes a TextStyle object.

- fontSize, fontWeight, and color are keys that define text appearance.
- Customization is done directly inside the widget instead of a separate stylesheet.

Summary of Fundamental Concepts

- Key-value pairs in Flutter allow customization and flexibility.
- They override default values to modify UI appearance & behavior.
- Flutter's named parameters improve readability and maintainability.
- Flutter's key-value structure resembles JSON but serves different purposes.

- Nesting widgets allows structuring UI elements clearly.
- Keeps related components together, making code more readable.
- Customization is easier by wrapping widgets inside style-defining widgets.
- Flutter UI is entirely built using a nested widget tree.

- Functions inside widgets define behavior (e.g., onPressed).
- Separating functions improves reusability, clarity, and maintainability.
- Properties inside widgets customize appearance (e.g., style in Text).
- Flutter combines UI and logic in one place, making code easier to manage.

Flutter and Dart Programming

Flutter is a framework that makes it easier to build apps with a beautiful user interface (UI) that works on both Android and iOS devices. **Dart** is the programming language behind Flutter. It's simple, fast, and designed to be easy to learn, especially for beginners.

Basic Concepts in Dart Programming

1. Dart variable
   - To store information like number or text.
   - To declare a variable, use types like var, int, double, String, etc.

```
var name = 'Alice';  // A string variable
int age = 30;        // An integer variable
```

2. Dart operators
   - Operators are used to perform operations on variables and values, like adding, subtracting, comparing, or assigning values.
   - For more operators information and how to use it, you can check here.

```
int sum = 5 + 3;     // Addition
bool isEqual = (5 == 5);  // Comparison
```

3. Dart keywords
   - Keywords are special words that have predefined meanings in Dart, such as if, for, class, void, etc. These words cannot be used as variable names.
   - For more keywords information, you can check here.
4. Dart Build in types (type data)
   - Dart supports several built-in types that allow you to work with different kinds of data:
     - ◊ Numbers (int, double)
       int for whole numbers, and double for decimal numbers.

```
int x = 10;
double y = 3.14;
```

     - ◊ Strings (String)
       Used to represent text.

```
String greeting = 'Hello, World!';
```

     - ◊ Booleans (bool)
       Represent `true` or `false`.

```
bool isActive = true;
```

     - ◊ **Records** ((value1, value2))
       A simple way to group multiple values.

```
var record = (10, 'hello');
```

◊ **Functions** (Function)
Functions perform actions and can return a results.

```
int add(int a, int b) {
   return a + b;
}
```

◊ **Lists** (List, also known as *arrays*)
Used to store multiple values in an ordered sequence.

```
List<String> colors = ['red', 'blue', 'green'];
```

- **.join()** method to combines all list elements into a single string. Can specify a custom separator (e.g., comma, space, hyphen) between elements.

```
List<String> fruits = ['Apple', 'Banana', 'Cherry'];
print(fruits.join(', ')); // Output: "Apple, Banana,
Cherry"
```

◊ **Sets** (Set)
A collection of unique items.

```
Set<int> uniqueNumbers = {1, 2, 3};
```

◊ **Maps** (Map)
A collection of key-value pairs.

```
Map<String, int> scores = {'Alice': 90, 'Bob': 85};
```

5. Dart Generic
   - Generics allow to write flexible code that works with different data types.
   - For example, you can create a list that holds only `String` values or only `int` values by using generics.

```
List<int> numbers = [1, 2, 3];
```

6. Dart Error Handling
   - Error handling helps manage and respond to problems that occur during the execution of your program.
   - In Dart, you use `try`, `catch`, and `finally` to handle errors.

```
try {
  int result = 10 ~/ 0;
} catch (e) {
  print('Cannot divide by zero: $e');
}
```

7. Dart OOP
   - Dart supports object-oriented programming, where you can create classes to represent objects.
   - These objects can have properties (variables) and methods (functions).

```
class Car {
  String make;
```

```
  String model;

  Car(this.make, this.model);

  void displayInfo() {
    print('Car: $make $model');
  }
}

void main() {
  var myCar = Car('Toyota', 'Corolla');
  myCar.displayInfo();
}
```

◊ Enum

An **enum** (short for "enumeration") is a special type in Dart that represents a fixed number of constant values.

Enums are helpful when you want to limit a variable to only accept specific values.

This makes your code safer by preventing invalid values from being assigned.

```
enum CarType { sedan, suv, truck }

void main() {
  CarType myCarType = CarType.sedan;

  if (myCarType == CarType.sedan) {
    print('Your car is a sedan.');
  }
}
```

8. Dart Null Safety
   - Null safety helps prevent errors caused by variables that are unexpectedly `null`.
   - In Dart, it is necessary to declare whether a variable can be null or not.

```
String? nullableString;  // This can be null
String nonNullableString = 'Hello';  // This cannot be
null
```

- Prevents accidental mistakes → Without `@override`, if the method is misspelled, Dart might not detect the error.

c. Comparison: `@override` in Java vs. Flutter
- Both Java and Flutter (Dart) use `@override` to indicate method overriding.
- This helps avoid mistakes when method signatures don't match.
  Java requires explicit method signatures, while Dart is more flexible.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- Explanation:
  - `@override` → Ensures `build()` in Child correctly overrides the parent's method.
  - If `build()` was misspelled, Java would throw an error.
- Comparison table :

Feature	**Java**	**Flutter**
Usage of `@override`	Required for overriding methods in OOP	Commonly used in Flutter widgets
Error Prevention	Prevents incorrect method signatures	Helps detect typos in overridden methods
Flexibility	Strict method signatures	More dynamic, but `@override` improves clarity

Step 6 : Exploring the Build Method and BuildContext

a. What is the `build()` Method?
- The `build()` method describes how a widget should appear on the screen.
- It returns a "widget tree", which is a collection of nested widgets forming the UI.
- The `build()` method is called whenever the UI needs to be updated.

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Hello, Flutter!'),
        ),
```

```
        ),
      );
    }
}
```

- Explanation:
  - `build()` returns a MaterialApp widget, which contains the entire app UI.
  - Inside MaterialApp, there are other widgets like Scaffold, Center, and Text.
  - This structure forms a widget tree, defining how UI elements are arranged.

b. What is BuildContext?
- BuildContext provides information about where a widget is located in the widget tree.
- It allows widgets to access inherited widgets and manage navigation.
- It is an essential parameter in the `build()` method.
- Without BuildContext, widgets wouldn't be able to access inherited properties.

c. How `build()` Keeps the UI Updated
- Whenever Flutter detects changes in the UI, it calls `build()`.
- The `build()` method ensures that the latest UI is displayed.
- Flutter's declarative UI approach means UI is rebuilt instead of manually modified.

d. Comparison with Java: How UI is Built
- In Java (Android), UI updates usually involve modifying View objects.
- In Flutter, UI is declarative and is rebuilt using `build()`.
- Flutter's approach makes UI updates more structured and predictable.

```
@Override
protected View build(Context context) {
    TextView textView = new TextView(context);
    textView.setText("Hello, Android!");
    return textView;
}
```

- Comparison Table :

Feature	**Java (Android)**	**Flutter (Dart)**
UI Structure	Uses XML or Java code for View	Uses build() method to return a widget tree
State Management	Manually updates UI views	UI is rebuilt when state changes
Flexibility	More complex, requires modifying View hierarchy	Simple, UI is updated automatically

Step 7 : What Does Return Do in Flutter

a. What Does return Do in build()?
- Inside the build() method, return specifies what the widget should display.
- It sends the widget back to Flutter for rendering on the screen.
- Flutter relies on return to construct and display UI elements dynamically.

b. Understanding the Returned Widgets

Widget	**Purpose**	**Example**